

Text Generation: Comparative Evaluation of Statistical and Neural Approaches

Knowledge Processing in Intelligent Systems (WiSe 2025/26)

Knowledge Technology, WTM

Department of Informatics, University of Hamburg

Aqsa Mohsin

aqsa.mohsin@studium.uni-hamburg.de

Laiba Qureshi

laiba.qureshi@studium.uni-hamburg.de

Abstract—Text generation is a fundamental task in Natural Language Processing (NLP) that involves learning patterns from text data in order to produce coherent sequences. This paper presents a comparative study of three text generation paradigms: statistical n-gram models, recurrent neural networks (RNNs) including Long Short-Term Memory (LSTM) networks, and transformer-based language models. All models are trained and evaluated on the same dataset, namely the Shakespeare text corpus provided by TensorFlow. The n-gram baseline shows that moving from unigrams to bigrams substantially reduces word-level perplexity, while trigram models suffer from sparsity, highlighting the limitations of fixed-context statistical modeling. Recurrent neural models significantly outperform statistical baselines, with the LSTM achieving the lowest character-level perplexity (2.67) and the lowest Self-BLEU scores, indicating the best balance between fluency and diversity on this dataset. A fine-tuned GPT-2 model outperforms the n-gram baseline and produces more globally coherent text, but exhibits higher perplexity and Self-BLEU than the LSTM. Overall, the findings highlight trade-offs between statistical, recurrent, and attention-based architectures, and show how architectural design, token granularity, and domain alignment influence fluency and diversity in text generation.

I. INTRODUCTION

Text generation models aim to learn the patterns of natural language so they can produce fluent, coherent sequences of text. At their core, these systems model language as a sequence prediction problem, estimating what comes next given the context. So their quality depends on how effectively they represent and use preceding information.

The objective of this project is to analyze how different model families handle language generation when trained on the same dataset. We compare statistical n-gram models, character-level recurrent neural networks, and a pretrained transformer model, focusing on how architectural choices and token granularity influence predictive performance and generative behavior, using perplexity to measure fluency and Self-BLEU to assess diversity across generated samples.

II. RELATED WORK

Early work on text generation relied on statistical methods such as Markov chains and n-gram language models. Shan-

non [6] demonstrated that simple probabilistic models could generate text with recognizable structure, and later work established word-level n-gram models as strong baselines for language modeling despite their limitations in handling long-range dependencies and data sparsity [1].

Recurrent neural networks (RNNs) introduced a major shift by enabling models to condition predictions on theoretically unbounded context. Mikolov et al. [4] showed that RNN language models outperform n-grams in perplexity, while gated variants such as LSTM [2] alleviate vanishing gradient issues and better capture long-term dependencies. Character-level RNNs have been shown to generate stylistically convincing Shakespeare-like text [3].

Transformer-based architectures replace recurrence with self-attention, enabling efficient modeling of long-range dependencies [8]. Pretrained transformer models such as GPT [5] demonstrate that large-scale pretraining followed by fine-tuning can achieve strong performance on generative tasks, motivating their inclusion in comparative studies of text generation.

Motivated by these line of works, we conduct a unified comparison of n-gram, character level RNN, and pretrained Transformer models on a Shakespeare corpus.

III. DATASET AND PREPROCESSING

All experiments use the Shakespeare corpus provided by TensorFlow [7], consisting of approximately 1.1 million characters and 65 unique symbols. Preprocessing is intentionally minimal to preserve the original structure and stylistic characteristics of the text. Dataset splits are constant as 80% Train, 10% as Validate, and 10% as Eval for all three implementations to maintain consistency and fair comparison.

IV. METHODOLOGY

This section describes the experimental pipeline used to compare the three generators introduced earlier.

A. Statistical n -gram Model

The statistical n -gram model serves as a baseline approach to text generation. It models the probability of a word given the preceding $n - 1$ words using frequency counts derived from the training corpus. Text generation proceeds by predicting the next word based on a fixed-length context window. In this project, a word-level backoff n -gram language model is implemented with add- k smoothing.

1) *Training Data Representation and Counting*: Training is performed on sentence-level text. Each sentence is tokenized into words and padded with $(n-1)$ start symbols $\langle s \rangle$ and an end symbol $\langle /s \rangle$. For each order $k \in \{1, \dots, n\}$, we count k -gram occurrences (c, w) and their context counts $C(c)$ over all sliding windows.

2) *Vocabulary Construction*: To reduce sparsity, a vocabulary is built from the training data. Tokens with frequency below the threshold `min_count` are mapped to $\langle \text{unk} \rangle$. This ensures consistent handling of rare and unseen tokens during both training and evaluation. In our experiments, we consider $n \in \{1, 2, 3\}$, use a fixed `min_count` threshold.

3) *Smoothing and Backoff Probability Estimation*: Probabilities are computed using add- k smoothing:

$$P(w | c) = \frac{C(c, w) + \alpha}{C(c) + \alpha|V|}, \quad (1)$$

where α is the smoothing constant and $|V|$ is the vocabulary size (including $\langle s \rangle$, $\langle /s \rangle$, and $\langle \text{unk} \rangle$). During inference, the model uses backoff: it attempts the highest-order n -gram with non-zero count and falls back to lower orders until unigrams.

4) *Text Generation*: Text is generated autoregressively from a seed prompt by repeatedly sampling the next token from the backoff distribution. We support greedy decoding (argmax) and temperature-controlled sampling; generation stops when $\langle /s \rangle$ is produced or a maximum length is reached.

B. Recurrent Neural Network Models (RNN and LSTM)

1) *Input Representation and Sequence Construction*: The Shakespeare corpus is processed at the character level. Each unique character is mapped to an integer index, forming a vocabulary of size $V = 65$. Training samples are generated using a sliding window approach with a fixed sequence length of 100 characters. Each input sequence consists of 100 consecutive characters, and the corresponding target sequence consists of the same sequence shifted by one character, resulting in a next-character prediction task.

2) *Model Architectures*: Two recurrent architectures are trained from scratch: a simple recurrent neural network (RNN) and a Long Short-Term Memory (LSTM) network. Both models share an identical architecture and differ only in the type of recurrent layer used. Each model consists of an embedding layer of dimension 128 that maps character indices to dense vectors, followed by a single recurrent layer with 256 hidden units. A fully connected output layer with

softmax activation produces a probability distribution over the character vocabulary for next-character prediction. Unlike the simple RNN, the LSTM incorporates gating mechanisms that regulate information flow across time steps, enabling more effective modeling of longer-range dependencies.

3) *Training Configuration*: Both models are trained using categorical cross-entropy loss for multi-class next-character prediction. Optimization is performed using the Adam optimizer with a learning rate of 0.001. Training is conducted using mini-batches of size 64. The maximum number of training epochs is set to 100. Early stopping based on training loss is employed with a patience of 4 epochs.

4) *Text Generation*: Text generation is performed autoregressively. A seed string is provided to the trained model, which produces a probability distribution over the next character at each time step. The next character is sampled from this distribution and appended to the input sequence, which is then shifted forward to generate subsequent characters.

C. Transformer-based Language Model (GPT-2)

1) *Tokenization and Fine-tuning*: For the transformer-based approach, a pretrained GPT-2 language model is fine-tuned with a causal language modeling objective, in which the model predicts the next token given all preceding tokens. The Shakespeare corpus is tokenized using GPT-2's byte-pair encoding (BPE) tokenizer and concatenated into a continuous subword token stream.

Training examples are created by splitting the token stream into contiguous 128-token blocks, with causal attention masks enforcing left-to-right conditioning. Optimization uses AdamW with a learning rate of 5×10^{-5} and weight decay of 0.01, along with early stopping based on validation loss. Fine-tuning is run for up to six epochs to adapt the model to the Shakespeare domain while limiting overfitting.

2) *Evaluation and Generation*: In addition to evaluation during fine-tuning, the GPT-2 model is evaluated on a short out-of-domain Wikipedia-style text sample. Perplexity is computed on this text to assess generalization beyond the Shakespeare domain. This Wikipedia text is used exclusively for evaluation and is not included in training.

Text generation with GPT-2 is performed by conditioning the model on a prompt and generating tokens autoregressively up to a maximum generation length of 200 tokens.

V. EVALUATION

Models are evaluated using

- **Perplexity (PPL)**: measures how well a language model predicts held-out text (fluency). *Lower* perplexity indicates better fit and typically more fluent output. Comparison shown in Fig 1
- **Self-BLEU**: measures diversity by scoring each generated sample against the rest as references. *Lower* Self-BLEU indicates more diverse generations. Comparison shown in Fig 2.

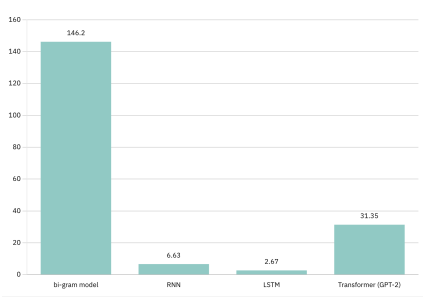


Figure 1: Perplexity comparison across text generation models evaluated at their respective token granularities (lower is better).

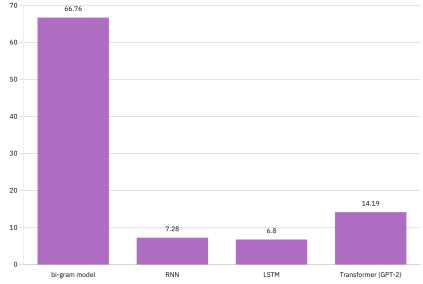


Figure 2: Self-BLEU comparison across models (lower indicates more diverse generations).

VI. RESULTS AND ANALYSIS

A. n -gram Perplexity and Backoff Diagnostics

Table I summarizes word-level perplexity and unigram fallback rates for backoff n -gram models on the Shakespeare corpus. Moving from unigrams ($n=1$) to bigrams ($n=2$) substantially reduces perplexity, indicating that short-range context captures strong local regularities in the text; however, bigrams still exhibit relatively high Self-BLEU scores compared to neural models (67.76, Fig. 2), reflecting limited diversity under fixed-context generation. In contrast, the trigram model ($n=3$) exhibits higher perplexity than the bigram baseline, suggesting that increased context length does not translate into better generalization under this data regime. A likely explanation is data sparsity at higher orders: many trigram contexts are rare or unseen, leading the model to rely more frequently on lower-order distributions despite smoothing. The fallback rate is similar for $n=2$ and $n=3$, reinforcing that sparsity remains a limiting factor for higher-order n -grams on this corpus.

This sparsity effect is also reflected in our backoff diagnostics as defined in Section IV-A3 and shown in (Fig. 3). First, the OOV-per-sentence histogram shows that a non-trivial subset of sentences contains one or more OOVs, which forces probability mass onto $\langle \text{unk} \rangle$ and weakens higher-order estimates (Fig. 3a). Second, the order-usage distribution confirms that the trigram model often attempts

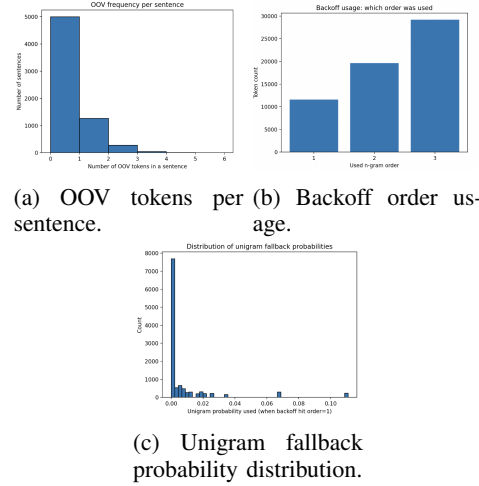


Figure 3: Backoff diagnostics for the n -gram model on the Shakespeare corpus.

Table I: Word-level perplexity (PPL) and unigram fallback rates for backoff n -gram models on the Shakespeare corpus. Lower PPL indicates better predictive performance.

Model	PPL	Fallback rate	Vocab
Unigram ($n = 1$)	264.10	1.000	5796
Bigram ($n = 2$)	146.20	0.190	5796
Trigram ($n = 3$)	238.91	0.190	5796

order-3 predictions but still falls back frequently to lower orders when contexts are missing (Fig. 3b). Finally, when backoff reaches order 1, the unigram probabilities used are heavily concentrated near zero with a long tail (Fig. 3c), indicating that many backed-off tokens are intrinsically low-frequency thus, the higher perplexity for $n=3$ under sparse contexts.

B. Neural Models & Transformer Training Behavior and Evaluation

This section analyzes training dynamics and evaluation metrics, including perplexity and Self-BLEU, for the neural text generation models: recurrent architectures (RNN and LSTM) and a transformer-based GPT-2 model.

For the recurrent models, training behavior is examined by monitoring training loss over epochs. Figure 4 shows the loss curves for the RNN and LSTM. The LSTM exhibits a rapid reduction in loss during the initial epochs, followed by steady convergence to a lower final value. In contrast, the simple RNN shows limited improvement after the early epochs and quickly plateaus at a higher loss, indicating less effective optimization under the same training setup.

On the held-out Shakespeare evaluation split, this difference in training behavior is reflected in perplexity. As shown in Table II, the simple RNN achieves a character-level perplexity of 6.63, while the LSTM attains a substantially lower value of 2.67, demonstrating the LSTM’s improved

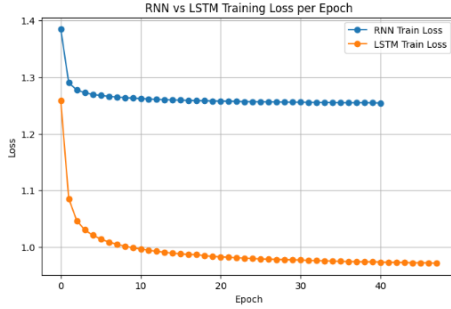


Figure 4: Training loss curves for the RNN and LSTM models.

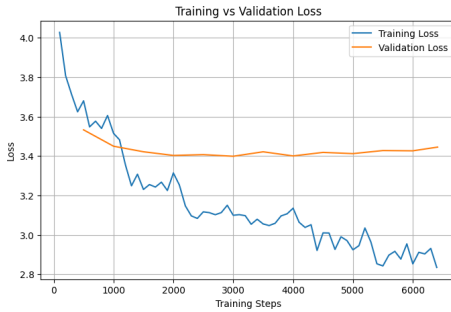


Figure 5: Training and validation loss during GPT-2 fine-tuning on Shakespeare.

ability to model character-level dependencies and assign higher probability to the correct next character.

For the transformer-based model, training behavior is evaluated by tracking both training and validation loss during fine-tuning on the Shakespeare corpus. As shown in Figure 5, training loss decreases steadily, while validation loss drops early and then stabilizes, suggesting that generalization performance converges quickly and that additional training yields diminishing returns.

Perplexity for GPT-2 is evaluated at the subword-token level. On the Shakespeare evaluation split, the fine-tuned model achieves a perplexity of 31.35. When evaluated on a short out-of-domain Wikipedia-style text sample, perplexity increases sharply to 194.45 (Table II). This substantial increase highlights the sensitivity of the transformer model to domain and stylistic mismatch between the fine-tuning data and the evaluation text.

In addition to perplexity, generation diversity is assessed using Self-BLEU scores, as shown in Figure 2. The n-gram model exhibits the highest Self-BLEU, indicating low diversity and substantial overlap across generated samples. The recurrent models achieve lower Self-BLEU values, reflecting more varied character-level generations, with the LSTM slightly outperforming the simple RNN. The GPT-2 model shows higher Self-BLEU than the recurrent models, suggesting increased repetition across samples under the chosen decoding settings.

Table II: Perplexity (PPL) results for neural and transformer-based models evaluated at their respective token levels.

Model	PPL	Eval. Set	Token Level
RNN	6.63	Shakespeare	Character
LSTM	2.67	Shakespeare	Character
GPT-2	31.35	Shakespeare	Subword
GPT-2	194.45	Wikipedia	Subword

This behavior indicates that, on this dataset, character-level recurrent models strike a more favorable balance between fluency and diversity than the transformer-based model. Despite its greater modeling capacity, GPT-2’s subword-level generation and decoding configuration lead to higher overlap across samples, highlighting that increased capacity does not necessarily result in more diverse text.

VII. CONCLUSION

This study presented a comparative evaluation of statistical and neural approaches to text generation using a common Shakespeare corpus. The n-gram experiments show that incorporating limited context improves performance over unigrams, while higher-order models suffer from data sparsity despite backoff mechanisms. Recurrent neural models further improve sequence modeling, with LSTM architectures achieving the lowest perplexity and the most stable training behavior among all evaluated models.

Generation diversity, measured using Self-BLEU, is also highest for the LSTM, indicating more diverse outputs than both the simple RNN and the transformer-based model. While GPT-2 produces longer and more globally coherent passages and outperforms the statistical baseline, it exhibits higher perplexity and Self-BLEU than the LSTM and degrades substantially on out-of-domain Wikipedia text.

Overall, the results show that increased model capacity alone does not guarantee improved fluency or diversity; architectural design, token granularity, and domain alignment are key determinants of text generation performance.

REFERENCES

- [1] Peter F. Brown, Vincent J. Della Pietra, Peter V. deSouza, Jenifer C. Lai, and Robert L. Mercer. Class-based n-gram models of natural language. *Computational Linguistics*, 18(4):467–480, 1992.
- [2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997.
- [3] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks, 2015.
- [4] Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Proceedings of Interspeech*, pages 1045–1048, 2010.
- [5] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.
- [6] Claude E. Shannon. Prediction and entropy of printed english. *Bell System Technical Journal*, 30(1):50–64, 1951.
- [7] TensorFlow. Shakespeare dataset, 2020. Accessed for character-level language modeling.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.